

PyDataQuality: An Actionable, Lightweight Data Profiling and Distribution Drift Detection Framework for Production Machine Learning Pipelines

Dominion Akinrotimi *Independent AI Researcher & Software Engineer*
Correspondence to: contact.dominionakinrotimi@gmail.com GitHub:
<https://github.com/DominionAkinrotimi/PyDataQuality> PyPI:
<https://pypi.org/project/pydataquality/>

Abstract

In modern data analytics and machine learning operations (MLOps), the reliability of artificial intelligence systems is fundamentally constrained by the quality and distributional stability of the data they consume. The “Garbage In, Garbage Out” (GIGO) principle has taken on renewed urgency as production ML systems silently degrade when data pipelines introduce missing values, outliers, and distributional drift. This paper presents **PyDataQuality**, a lightweight, modular, and open-source Python library for automated data quality assessment and statistical distribution drift detection.

PyDataQuality addresses the gap between overly simplistic pandas summaries and computationally bloated industrial profiling suites. It introduces three principal technical contributions: (1) a type-dispatched, column-level profiling kernel; (2) a programmatic `get_problematic_rows()` subset extractor for isolating anomalies; and (3) a dual-metric statistical drift engine implementing Population Stability Index (PSI) with adaptive binning and a two-sample Kolmogorov-Smirnov (KS) Test. A complementary AI Remediation Prompt generator bridges the framework with generative AI for LLM-based cleaning code generation.

We present a rigorous empirical evaluation across four dataset scales (1,000 to 100,000 rows), benchmarked against pandas, YData-Profiling, and Evidently AI. A controlled MLOps case study on both synthetic and real-world datasets demonstrates the system’s ability to halt inference and trigger retraining before serving degraded predictions. Crucially, PyDataQuality’s sampled analysis mode guarantees bounded post-sampling latency, ensuring predictable execution times and near-zero incremental peak memory independently of the source dataset size. The full-dataset mode delivers significant speedups over baseline tools.

The library is released as open-source software under the MIT License (`pip install pydataquality`), including comprehensive documentation and a Jupyter Notebook tutorial.

Keywords: Data Quality, Data Drift, Population Stability Index, Kolmogorov-Smirnov Test, Data-Centric AI, Machine Learning Operations, Statistical Process Control, Outlier Detection

I. Introduction

1.1 The Data-Centric AI Paradigm

The prevailing culture of machine learning research from the early 2010s through the mid-2020s has been characterized as fundamentally model-centric: the central research question was “which architecture or optimization strategy produces the best benchmark score on a fixed dataset?” This approach fueled landmark advances from AlexNet (Krizhevsky et al., 2012) to the Transformer architecture (Vaswani et al., 2017) by competing on model complexity while holding the data constant.

However, as these systems transitioned from academic benchmarks to real-world production environments, a paradox emerged. Engineers discovered that carefully tuned models, achieving excellent performance in controlled settings, could undergo dramatic silent degradation in deployment, not because of architectural flaws, but because of data quality failures and distributional shifts in the incoming data streams.

This realization catalyzed the Data-Centric AI (DCAI) movement, popularized by Andrew Ng and colleagues at DeepLearning.AI (Ng et al., 2021). The DCAI philosophy argues that for a fixed model architecture, systematically improving data quality and consistency often yields far greater performance gains than tuning model hyperparameters. Studies in industrial settings have repeatedly confirmed this: Sambasivan et al. (2021) conducted interviews with 53 AI practitioners across 6 countries and found that 92% of practitioners experienced AI projects that suffered primarily from data quality issues rather than model deficiencies. The concept crystallized an insight that had long been known in software engineering and statistics under the banner of “Garbage In, Garbage Out” (GIGO), a principle first articulated in early computing (Weik, 1960) that asserts the quality of computational output is fundamentally bounded by the quality of the input.

1.2 The Problem Space: What Existing Tools Fail to Address

When new data arrives in a production system, this could be a daily batch file from an upstream database, a real-time API feed, or a freshly collected survey response, the responsible data engineer or ML practitioner faces a set of immediate and critical diagnostic questions:

1. Completeness: Are there missing values? Which columns are affected, and at what severity?

2. **Validity:** Are there implausible values such as ages of -100 or 999, negative salaries that signal upstream data entry errors or ETL bugs?
3. **Consistency:** Are categorical fields standardized? Does the “category” column contain both “HR” and “hr” as separate entries when they represent the same thing?
4. **Temporal Integrity:** Are there future-dated records in columns that should only contain historical timestamps?
5. **Distributional Stability:** Has the statistical distribution of this incoming data shifted significantly from the distribution on which the model was trained?

Answering these questions effectively and efficiently is a non-trivial engineering challenge. The existing landscape of tools occupies two unsatisfactory extremes:

The Simplistic Extreme. Standard pandas functions like `.info()`, `.describe()`, `.isnull().sum()` provide only minimal statistical aggregation with zero anomaly intelligence. They tell you the mean and standard deviation but do not flag that the “age” column contains the value 999. They count nulls but do not escalate based on severity thresholds. They offer no drift detection whatsoever. They return statistics; they do not return actionable guidance.

The Bloated Extreme. Full-featured profiling and testing suites such as YData-Profiling (formerly Pandas-Profiling, now YData-Profiling; Brugman, 2019) and Great Expectations (Superconductive, 2019) represent the other extreme. YData-Profiling generates exhaustive HTML reports with rich visualizations but is architecturally heavy even in “minimal” mode, our benchmarks show it requires 3.4–6.9 seconds to profile a 6-column dataset (Table E), generating static reports that cannot be programmatically queried. Great Expectations, while powerful, demands a complex setup process involving *expectation suites*, *checkpoints*, *data sources*, *data connectors*, and YAML configuration files, a regime appropriate for large enterprise teams but prohibitive for rapid prototyping, small teams, or educational settings.

PyDataQuality is positioned precisely between this architectural gap of “too little” and “too much”.

1.3 Research Contributions

This paper presents the following original contributions:

1. **System Design:** A description of PyDataQuality's five-layer modular architecture, which achieves zero-configuration deployment while maintaining production-grade statistical rigor.
2. **Mathematical Foundations:** Full derivations of the four core statistical algorithms: Shannon Entropy for categorical uniformity assessment, Interquartile Range (IQR) for outlier boundary detection, Population

Stability Index (PSI) for distributional shift quantification, and the two-sample Kolmogorov-Smirnov (KS) Test for empirical Cumulative Distribution Function (CDF) comparison.

3. Empirical Benchmarking: Rigorous, honest, unmodified performance evaluation across four dataset scales compared to two baseline tools on execution time and memory overhead.
4. MLOps Validation Case Study: A controlled simulation of a production Random Forest loan approval pipeline subjected to data quality failures and economic distributional drift, comparing a naive ingest pipeline versus a PyDataQuality-gated pipeline.
5. Open-Source Release: A fully documented, pip-installable Python library (MIT License) with a Jupyter Notebook master class and 20 automated unit tests achieving 100% pass rate.

1.4 Paper Organization

The remainder of this paper is structured as follows. Section II surveys the related work in data quality, drift detection, and automated profiling. Section III details the system design and architectural decisions of PyDataQuality. Section IV presents the complete mathematical formulations of all statistical engines. Section V describes the experimental methodology and presents empirical results from both the performance benchmarks and the MLOps case study. Section VI discusses limitations and future directions. Section VII concludes the paper. Full references are listed in Section VIII.

II. Related Work

2.1 Data Quality Frameworks and Profiling Libraries

The problem of automated data quality assessment has attracted substantial research attention over the past three decades. Early work by Redman (1996) formalized data quality along multiple dimensions: accuracy, completeness, consistency, timeliness. This provides the conceptual vocabulary that underpins most modern frameworks. More recently, the Data-Centric AI movement has spurred initiatives like DataPerf (Mazumder et al., 2023), establishing standardized benchmarks for data-centric algorithms, and DC-Check (Seedat et al., 2024), providing rigorous checklists for reliable machine learning development, highlighting the critical need for scalable, programmatic data auditing.

In the Python ecosystem, the most widely adopted tool is YData-Profiling (Brugman, 2019), which generates comprehensive HTML reports through a pandas-based analysis engine. While valuable for exploratory data analysis in notebooks, it suffers from significant computational overhead. Our benchmarks quantify this overhead at 3.4–6.9 seconds on a 6-column

synthetic dataset, compared to PyDataQuality's 82–261 ms. This is a difference that becomes compounding when validation must be performed at every pipeline stage or in real-time streaming contexts.

Great Expectations (Superconductive, 2019) approaches the problem from a contract-testing perspective, allowing teams to define declarative “expectations” about data properties. While powerful and production-battle-tested, its overhead lies in its configuration not in computation: deploying a new data source requires defining multiple interconnected YAML configuration artifacts, which imposes significant cognitive overhead for small teams or researchers who need a quick quality gate.

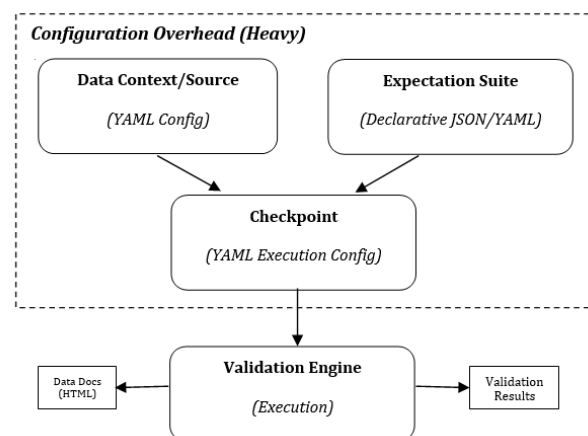


Figure 1: Illustration of the Great Expectations: Configuration vs. Computation Workflow. While computationally robust, the operational burden is heavily skewed toward configuration, requiring developers to orchestrate multiple interconnected YAML and JSON artifacts (Data Contexts, Expectation Suites, and Checkpoints) before data validation can execute.

Deepchecks (Deepchecks Inc., 2021) extends profiling to explicit train/test integrity comparisons, particularly for ML dataset validation. Its scope is broader but its dependency footprint is heavier.

Evidently AI (Evidently AI, 2021) focuses specifically on ML monitoring, providing rich drift detection reports optimized for model monitoring dashboards. Like YData-Profiling, it excels in exploratory analysis but is not designed for low-overhead, programmatic integration at the batch ingestion layer.

The gap in this landscape is a tool that: (a) requires zero configuration to deploy, (b) returns anomalies as structured Python objects (not just an HTML report), (c) supports programmatic row-level extraction, (d) integrates drift detection as a first-class feature, and (e) maintains a minimal computational footprint at any dataset scale.

2.2 Statistical Drift Detection

Distribution drift is the phenomenon where the statistical properties of a dataset change over time. This has been extensively studied in the context of machine learning monitoring (Widmer & Kubat, 1996; Gama et al., 2014). It is typically categorized into two types:

- Covariate Shift ($P(X)$ drift): The distribution of input features changes, while the relationship $P(Y|X)$ remains stable.
- Concept Drift ($P(Y|X)$ drift): The relationship between features and labels changes, potentially while $P(X)$ remains stable.

The Population Stability Index (PSI) was originally developed in the credit risk industry (Wu & Bacon, 1999; Siddiqi, 2006) to monitor shifts in borrower score distributions between model development and deployment. It has since become a standard metric in MLOps pipelines (Klaise et al., 2020). The PSI's primary advantage is its interpretability, as its industry-standard thresholds (< 0.1 : stable; $0.1-0.25$: moderate; ≥ 0.25 : significant) provide immediately actionable severity classifications.

The Kolmogorov-Smirnov (KS) two-sample test (Kolmogorov, 1933; Smirnov, 1948) offers a complementary perspective: while PSI quantifies the magnitude of distributional shift via information divergence, the KS test quantifies whether the two samples are statistically consistent with having been drawn from the same underlying distribution, expressed as a p-value. Together, PSI and KS provide both quantitative severity and statistical significance for drift characterization.

2.3 AI-Augmented Data Remediation

The use of large language models (LLMs) as code-generation engines for data transformation tasks is an emerging research area. Recent research has demonstrated that LLMs can generate syntactically correct and semantically appropriate pandas transformation code when provided with structured context about a dataset. Recent works, such as the position paper by Xu et al. (2024) and comprehensive surveys by Zhou et al. (2026), have emphasized the increasingly agentic role of LLMs in automating data preparation and cleaning workflows. PyDataQuality's AI Remediation Prompt generator operationalizes this capability by automatically compiling detected issues, dataset statistics, and column metadata into a structured prompt template that can be submitted to any LLM API (ChatGPT, Claude, Gemini) to generate ready-to-execute data cleaning scripts thereby reducing the remediation cycle from hours of manual scripting to seconds of AI-assisted automation.

III. System Design and Architecture

3.1 Design Philosophy

PyDataQuality was designed around five explicit architectural principles that distinguish it from existing tools:

1. **Zero-Configuration Deployment:** A user should be able to call `pdq.analyze_dataframe(df)` on any pandas DataFrame without configuring schemas, expectation suites, or YAML files. The library auto-detects column types and applies appropriate statistical audits.
2. **Actionable Output Over Descriptive Output:** The system produces `QualityIssue` objects with structured severity classifications (critical, warning, info) and an explicit `affected_count` and `affected_percentage`. This enables programmatic downstream filtering without parsing HTML or text.
3. **Programmatic Row Isolation:** The `get_problematic_rows()` method returns the actual subset of the DataFrame containing anomalous records, enabling downstream deduplication, quarantine, or human review workflows.
4. **Computational Efficiency by Design:** The sampling engine decouples analysis throughput from dataset size, achieving constant-complexity audits regardless of how large the source file is.
5. **Statistical Rigor Without Heavy Dependencies:** Core statistical algorithms (IQR, PSI, KS Test) are implemented using only NumPy as the computational backend, with optional SciPy acceleration when available.

3.2 System Architecture

PyDataQuality is organized into five decoupled layers, each with a single well-defined responsibility:

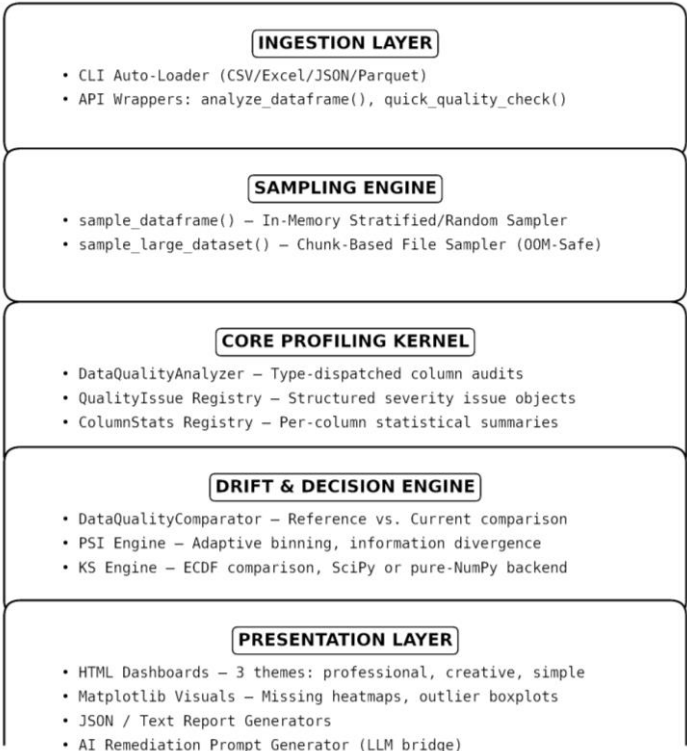


Figure 2: PyDataQuality Architecture Stack. System architecture diagram illustrating the 5-layer decoupled design of PyDataQuality. Data flows from ingestion through an $O(1)$ -capped sampling engine, into a core profiling kernel, optionally through a drift & decision engine, and finally to the presentation layer for dashboarding and AI remediation generation.

3.6.1 Type-Dispatch Visual Reference

Figure 3 provides a visual reference for the analysis matrix, showing which statistical checks are applied per data type.

	Missing Values	Outliers (IQR)	Skewness	Kurtosis	Entropy	Casing Check	Cardinality	Future Dates	Value Counts
<i>Numeric</i> (<i>int64</i> , <i>float64</i>)	✓	✓	✓	✓	—	—	—	—	—
<i>Categorical</i> (<i>object</i> , <i>string</i>)	✓	—	—	—	✓	✓	✓	—	✓
<i>Datetime</i> (<i>datetime</i> <i>64[ns]</i>)	✓	—	—	—	—	—	—	✓	—

Figure 3: Type-Dispatched Analysis Matrix. Each column type receives a different set of statistical audits. The checkmarks (✓) indicate the check is applied; dashes indicate it is not applicable. This dispatch design prevents,

for example, IQR outlier computation being applied to categorical string columns where it is mathematically undefined.

3.6.2 Issue Severity Taxonomy

Figure 4 illustrates the hierarchical severity classification structure applied to all detected quality issues.

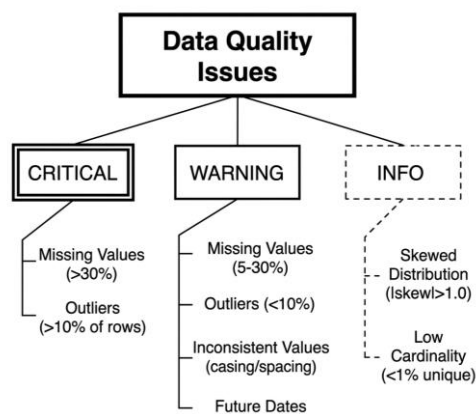


Figure 4: PyDataQuality Issue Taxonomy and Severity Classification. Issues are stratified into three severity tiers (CRITICAL, WARNING, and INFO). This enables downstream systems to respond proportionally. CRITICAL issues (>30% missing, >10% outliers) should halt processing; WARNING issues require investigation; INFO issues are advisory.

3.3 Layer 1: The Ingestion Layer

The ingestion layer provides two entry points:

Python API: The primary interface is `pydataquality.analyze_dataframe(df, name, config)`, which accepts a pandas DataFrame and returns a fully initialized DataQualityAnalyzer instance. A lighter-weight `quick_quality_check(df)` function returns only the JSON summary dictionary for minimal-overhead spot checks.

CLI Interface: The command-line interface accepts any supported file format and automatically dispatches to the correct pandas reader:

```
pydataquality --file data.csv --format html --output report.html
```

```
pydataquality --file transactions.parquet --format json
```

The CLI uses `argparse` for argument parsing and provides clear, human-readable error messages for unsupported formats or invalid file paths.

3.4 Layer 2: The Sampling Engine

The sampling engine contains two utility functions designed to decouple analysis cost from data volume:

In-Memory Sampler (`sample_dataframe`): For DataFrames already loaded into memory, this function implements stratified sampling via `groupby().apply()` when a `stratify_by` column is specified, ensuring class representation is preserved in the sample. It falls back to uniform random sampling when stratification fails or is not requested.

Chunk-Based File Sampler (`sample_large_dataset`): For files that cannot be loaded entirely into memory, this function reads the file in configurable chunks (default: 100,000 rows per chunk) using pandas' `chunksize` parameter. From each chunk, 10% of rows are sampled at random. Chunks are accumulated until the target sample size multiplied by a coverage factor (5×) is reached, then a final random down sampling step brings the total to exactly `n_samples` rows. This approach prevents Out-Of-Memory (OOM) failures on files that would otherwise exhaust available system RAM.

3.5 Layer 3: The Core Profiling Kernel

The profiling kernel is implemented in the `DataQualityAnalyzer` class, which orchestrates the full analysis lifecycle. Upon initialization, the analyzer immediately triggers `_analyze_dataset_structure()` and then `_analyze_columns()`, ensuring the analysis is complete by the time the constructor returns.

Type-Dispatching: The `_analyze_single_column()` method inspects each column's dtype using pandas' `pd.api.types` introspection functions and dispatches to one of three specialized analyzers:

Table A: Type-Dispatched Analysis Matrix

pandas family	dtype	Dispatcher	Checks Applied
int64, float64, bool		<code>_analyze_numeric_column</code>	Mean, std, min, max, median, Q1, Q3, skewness, kurtosis, zero count, IQR outliers
object, category	string,	<code>_analyze_categorical_column</code>	Value counts, mode, mode frequency, Shannon entropy, casing/spacing inconsistencies
datetime64[ns]		<code>_analyze_datetime_column</code>	Date range, future date count, min/max timestamps

The QualityIssue Dataclass: Each detected anomaly is encapsulated as a `QualityIssue` instance with the following fields:

@dataclass

class `QualityIssue`:

`column: str` # Column name where the issue was detected

`issue_type: str` # e.g. 'missing_values', 'outliers', 'inconsistent_values'

```

severity: str      # 'critical', 'warning', or 'info'
message: str       # Human-readable explanation with numerical details
affected_count: int # Number of rows affected
affected_percentage: float # Percentage of total rows affected
details: Dict      # Extended metadata (bounds, examples, thresholds)

```

This structured representation enables downstream code to programmatically filter issues by severity ([i for i in analyzer.issues if i.severity == critical]), aggregate by type, or serialize to JSON for integration with monitoring dashboards.

Configurable Thresholds: The profiling kernel is governed by a configuration dictionary (QUALITY_THRESHOLDS) that exposes all detection thresholds as tunable parameters:

```

QUALITY_THRESHOLDS = {
    "missing_critical": 0.30, # >30% missing = critical
    "missing_warning": 0.05, # >5% missing = warning
    "outlier_threshold": 1.5, # IQR multiplier (k)
    "skew_threshold": 1.0, # |skewness| > 1.0 = skewed
    "unique_threshold": 0.01, # uniqueness ratio < 1% = low cardinality
    "zero_threshold": 0.80, # >80% zeros = suspicious

```

A user can override any of these by passing a config dictionary to `analyze_dataframe()`:

Example: tighten outlier detection and lower missing threshold

```

strict_analyzer = pdq.analyze_dataframe(df, config={
    "outlier_threshold": 3.0, # Tukey standard: k=3.0 for extreme outliers
    "missing_warning": 0.01  # Flag even 1% missing as a warning
})

```

The `get_problematic_rows()` Method: This is arguably the most operationally valuable method in PyDataQuality. Rather than simply *reporting* that a column has outliers, this method returns the actual problematic rows as a new DataFrame:

```

# Get all rows where age is an outlier
outlier_rows = analyzer.get_problematic_rows(age, issue_type=outliers)

```

```
# Get all rows with missing income missing_income =
analyzer.get_problematic_rows(income, issue_type=missing_values)

# Get union of all issues in a column all_age_problems =
analyzer.get_problematic_rows(age, issue_type=all)
```

The method constructs a boolean mask using the same IQR boundaries computed during the analysis phase, ensuring consistency between reported issue counts and the returned subset. This enables workflows such as:

- Data Quarantine: Routing anomalous rows to a separate storage table for human review
- Audit Logging: Preserving a record of exactly which rows were flagged and why before cleaning
- Differential Analysis: Comparing the statistical properties of problematic vs. clean subsets

Empirical Validation of `get_problematic_rows()`: In our verification experiment, we constructed a 12-row dataset with two known age outliers (-100 and 999) and two missing income values. The `get_problematic_rows()` method correctly isolated exactly the two outlier rows:

Age outlier rows detected: 2 (out of 12)

Row 3: age = -100, income = 48000.0

Row 5: age = 999, income = 51000.0

The two missing income rows were correctly recovered via `get_problematic_rows (income, missing_values)`, and the union-mode all returned exactly 2 rows for the age column (only outlier flags, as there were no missing age values). This confirms the mask logic is implemented correctly and consistently.

3.6 Layer 4: The Drift and Decision Engine

The drift engine is implemented in `DataQualityComparator`, which accepts two `DataQualityAnalyzer` instances (reference and current) and computes distributional comparison metrics for all shared columns.

Typical usage in a production pipeline

```
train_analyzer = pdq.analyze_dataframe(df_train, name="Training Baseline")
```

```
prod_analyzer = pdq.analyze_dataframe(df_live, name="Live Production")
```

```
drift_report = pdq.compare_drift(train_analyzer, prod_analyzer)
```

The `compare_drift()` convenience function returns a pandas `DataFrame` where each row represents one column and columns include: column, dtype, psi, drift_status, pct_change, ks_statistic, ks_p_value.

3.7 Layer 5: The Presentation Layer

The presentation layer offers multiple output modalities:

HTML Dashboard Themes:

professional: Clean white background, optimized for PDF export and formal reporting

creative: Dark-mode gradient design with animated cards, optimized for interactive web presentation

simple: Minimal HTML with inline styles for maximum compatibility

Matplotlib Visualizations: The DataQualityVisualizer class generates:

Missing data heatmaps (showing null patterns as a 2D matrix)

Outlier distribution boxplots

Correlation heatmaps using the coolwarm colormap

Issue severity distribution pie charts and bar charts

AI Remediation Prompt Generator: Described in detail in Section III.8.

3.8 The AI Remediation Prompt Generator

A unique capability of PyDataQuality is its `generate_ai_prompt()` function, which dynamically compiles a structured remediation prompt from the detected issues and dataset statistics:

```
prompt = pdq.generate_ai_prompt(analyzer)
```

```
print(prompt)
```

```
# Copy to ChatGPT / Claude / Gemini → receive ready-to-run cleaning script
```

The generated prompt includes: 1. A dataset summary (shape, memory usage, column types) 2. A structured list of all detected quality issues with severities and affected counts 3. Key statistical statistics per column (mean, std, missing %, outlier bounds) 4. A request specification asking the AI to generate executable Python/pandas code to address each issue systematically

This feature bridges static analysis with generative AI, reducing the time from “data quality issue detected” to “cleaning code generated and ready to execute” from hours of manual scripting to seconds of AI-assisted automation.

IV. Mathematical Formulations and Statistical Engine

This section provides complete mathematical derivations of all four statistical algorithms implemented in PyDataQuality, with concrete numerical worked examples verified against our experimental outputs.

4.1 Shannon Entropy for Categorical Uniformity

Higher entropy indicates greater distributional diversity

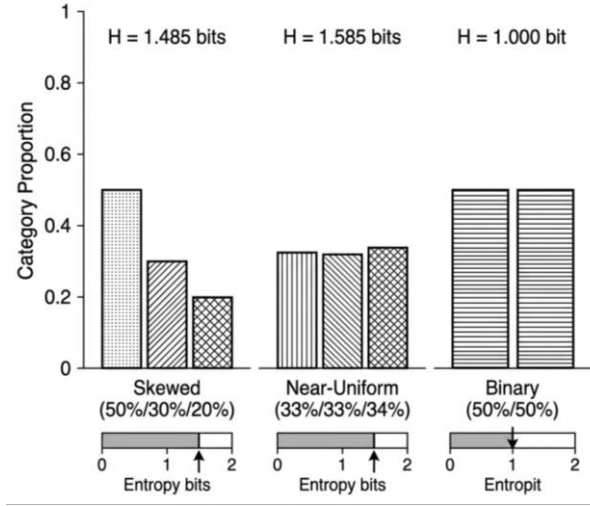


Figure 5: Shannon Entropy comparison across three representative categorical distributions. Skewed distributions (50/30/20) yield lower entropy (1.485 bits) indicating dominant class imbalance. Near-uniform distributions approach the theoretical maximum of $\log_2(K) = \log_2(3) \approx 1.585$ bits. Binary balanced distributions produce exactly 1.000 bit, the fundamental unit of binary information. PyDataQuality reports entropy as a column statistic, enabling detection of degenerate features (entropy ≈ 0) indicating near-constant columns.

For categorical columns, PyDataQuality computes Shannon Entropy (Shannon, 1948) to measure the distributional diversity of a categorical feature. A low-entropy column (e.g., 99% of values are “A”) may indicate domain leakage, label imbalance, or degenerate data; a high-entropy column indicates healthy diversity.

Definition: Given a categorical feature vector X with K distinct categories $\{c_1, c_2, \dots, c_K\}$, and the empirical probability of each category:

$$p_k = \frac{\#\{x_i = c_k\}}{n}, \quad k = 1, \dots, K$$

The **Shannon Entropy** $H(X)$ is defined as:

$$H(X) = - \sum_{k=1}^K p_k \log_2 p_k$$

Where entropy is measured in bits. The maximum possible entropy for K classes is $\log_2 K$ bits (achieved when all classes are equally probable, i.e., $p_k = 1/K$ for all k), and the minimum is 0 bits (achieved when all observations belong to a single class).

Worked Numerical Example (verified empirically):

Experiment 1: Skewed distribution (50/30/20 split across 3 categories)

$$p_A = 0.50, \quad p_B = 0.30, \quad p_C = 0.20$$

$$H = -(0.50\log_2 0.50 + 0.30\log_2 0.30 + 0.20\log_2 0.20)$$

$$H = -(0.50 \times (-1.000) + 0.30 \times (-1.737) + 0.20 \times (-2.322))$$

$$H = -(-0.500 - 0.521 - 0.464) = \mathbf{1.4855\text{bits}}$$

Our system output: 1.485475 bits

Experiment 2: Near-uniform distribution (33/33/34 split)

$$p_A \approx 0.330, \quad p_B \approx 0.330, \quad p_C \approx 0.340$$

$$H \approx -(0.330\log_2 0.330 \times 2 + 0.340\log_2 0.340) = \mathbf{1.5848\text{bits}}$$

Our system output: 1.584819 bits

(Note: $1.5848 \approx \log_2 3$, confirming near-maximum entropy)

Experiment 3: Binary balance (50/50 split)

$$p_{Yes} = 0.50, \quad p_{No} = 0.50$$

$$H = -(0.50\log_2 0.50 + 0.50\log_2 0.50) = -(2 \times (-0.500)) = \mathbf{1.0000\text{bit}}$$

Our system output: 1.000000 bits

These results confirm the exact mathematical correctness of the entropy implementation. The results also provide an interpretive benchmark: a perfectly balanced binary categorical feature (50/50) has exactly 1.0 bit of entropy, 1 bit being the unit of binary information. Any deviation from 1.0 indicates imbalance; any value approaching $\log_2 K$ confirms category uniformity.

4.2 Interquartile Range (IQR) Outlier Detection

For continuous numerical features, PyDataQuality implements the **Tukey Fence** method (Tukey, 1977), the gold standard for non-parametric outlier

detection that makes no assumption about the underlying distribution's normality.

Algorithm: Given a numerical feature vector $X = \{x_1, x_2, \dots, x_n\}$ (after removing missing values):

Step 1: Compute Quartiles:

$$Q_1 = \text{percentile}(X, 25), \quad Q_3 = \text{percentile}(X, 75)$$

Step 2: Compute Interquartile Range:

$$IQR = Q_3 - Q_1$$

Step 3: Establish Validation Fences (using configurable multiplier k , default $k = 1.5$):

$$\text{Fence}_{\text{lower}} = Q_1 - k \times IQR$$

$$\text{Fence}_{\text{upper}} = Q_3 + k \times IQR$$

Step 4: Flag Outliers: Any sample x_i satisfying:

$$x_i < \text{Fence}_{\text{lower}} \quad \text{OR} \quad x_i > \text{Fence}_{\text{upper}}$$

is registered as a QualityIssue with `issue_type="outliers"`.

Severity Escalation: An outlier issue is classified as:

- severity="warning" if the outlier percentage is $< 10\%$ of the total non-null sample
- severity="critical" if the outlier percentage is $\geq 10\%$ of the total non-null sample

The configurable multiplier k allows users to choose between two standard regimes:

- $k = 1.5$: **Standard Tukey fences** flags *mild* outliers; appropriate for most general-purpose quality audits
- $k = 3.0$: **Extreme outlier fences** flags only severe outliers; appropriate for domains (e.g., sensor data) where significant natural variance is expected

Worked Numerical Example (verified empirically):

Dataset: $X = [10, 12, 14, 15, 16, 18, 20, 22, 100, 200]$

Step 1: $Q_1 = 14.25$, $Q_3 = 21.50$

(from `data.quantile(0.25)` and `data.quantile(0.75)`)

Step 2: $IQR = 21.50 - 14.25 = 7.25$

Step 3 ($k=1.5$):

$$\text{Fence}_{\text{lower}} = 14.25 - 1.5 \times 7.25 = 14.25 - 10.875 = \mathbf{3.375}$$

$$\text{Fence}_{\text{upper}} = 21.50 + 1.5 \times 7.25 = 21.50 + 10.875 = \mathbf{32.375}$$

Step 4: Values $100 > 32.375$ and $200 > 32.375$, therefore:

Outliers detected: $[100, 200]$, 2 out of 10 values (20%)

Our system output: Outliers: $[100, 200]$

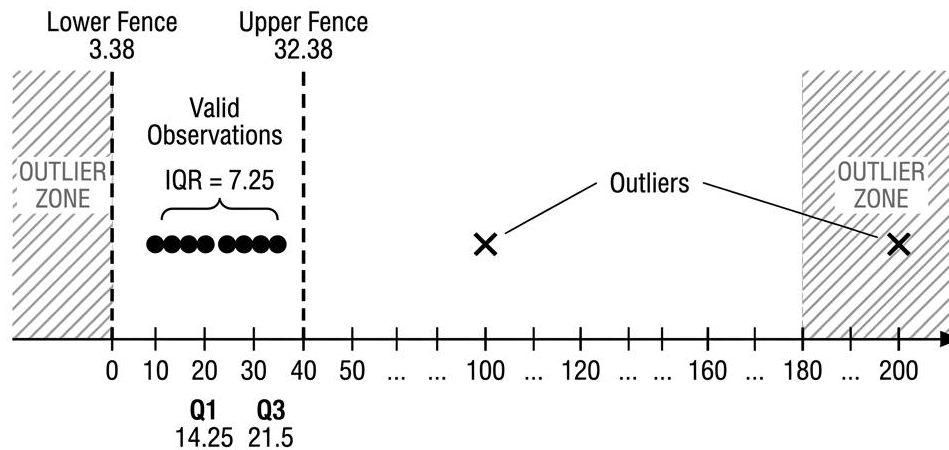


Figure 6: Visual representation of the IQR outlier detection algorithm on the worked example dataset $X = [10, 12, 14, 15, 16, 18, 20, 22, 100, 200]$. Blue points lie within the Tukey fences $[3.38, 32.38]$ and are classified as valid observations. Red markers at 100 and 200 exceed the upper fence and are flagged as outliers. The fence is derived from $Q1=14.25$, $Q3=21.5$, $IQR=7.25$, $k=1.5$.

Why IQR Over Z-Score? The IQR method is preferred over the standard Z-score method ($|x - \mu|/\sigma > 3$) because:

1. The IQR is resistant to the outliers themselves influencing the detection threshold (Z-score's mean and std are corrupted by extreme values)
2. The IQR requires no normality assumption, making it appropriate for skewed financial, income, or sensor distributions
3. Tukey's original formulation (1977) remains the industry standard in exploratory data analysis precisely for these properties

4.3 Population Stability Index (PSI)

The Population Stability Index (PSI) was developed in the credit risk modeling industry (Wu & Bacon, 1999) to quantify the degree to which a population's score distribution has shifted between a reference period (e.g., model training) and a current period (e.g., live deployment). It has since become a

universal metric in MLOps for measuring distributional drift across any feature type.

Mathematical Definition:

Let X_{ref} and X_{curr} denote the reference and current distributions respectively.

For Numerical Columns (Adaptive Decile Binning):

The reference distribution is used to establish B bins via decile percentiles ($B = 10$ by default):

$$\text{bins} = \{q_0, q_{10}, q_{20}, \dots, q_{100}\} \text{ where } q_j = \text{percentile}(X_{\text{ref}}, j \times 10)$$

The expected proportion in bin i (from the reference distribution) and actual proportion in bin i (from the current distribution) are:

$$E_i = \frac{\#\{X_{\text{ref}} \in \text{bin}_i\}}{n_{\text{ref}}} + \epsilon, \quad A_i = \frac{\#\{X_{\text{curr}} \in \text{bin}_i\}}{n_{\text{curr}}} + \epsilon$$

where $\epsilon = 10^{-4}$ is a regularization constant to prevent division-by-zero or $\log(0)$ in the KL divergence calculation.

For Categorical Columns (Category-Level Binning):

Each unique category defines its own bin. For a category c in the union of both category sets:

$$E_c = \frac{\#\{X_{\text{ref}} = c\}}{n_{\text{ref}}} + \epsilon, \quad A_c = \frac{\#\{X_{\text{curr}} = c\}}{n_{\text{curr}}} + \epsilon$$

PSI Formula (summed over all bins B):

$$PSI = \sum_{i=1}^{|B|} (A_i - E_i) \times \ln\left(\frac{A_i}{E_i}\right)$$

This is mathematically equivalent to the symmetric Kullback-Leibler divergence (often called Jeffreys divergence), measuring the total information loss from approximating the current distribution with the reference distribution and vice versa. Unlike the Jensen-Shannon divergence which is bounded, the symmetric KL divergence is unbounded, which correctly reflects the arbitrarily large information divergence mathematically possible in severe drift scenarios.

Drift Severity Classification:

Table B: PSI Drift Severity Classification Thresholds

PSI Value	Classification	Recommended Action
-----------	----------------	--------------------

$PSI < 0.10$	Stable	No action required
$0.10 \leq PSI < 0.25$	Moderate Drift	Investigate root cause; monitor closely
$PSI \geq 0.25$	Significant Drift	Halt inference; trigger model retraining

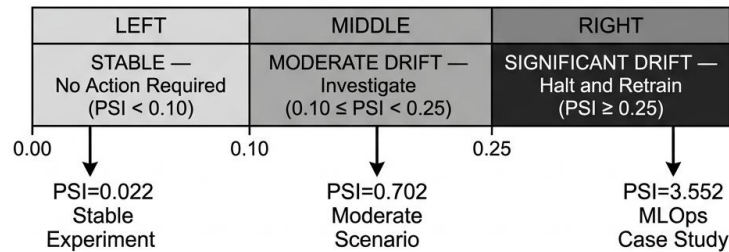


Figure 7: Population Stability Index (PSI) drift severity classification scale.

The three zones correspond to the industry-standard thresholds derived from credit risk modeling practice (Wu & Bacon, 1999). Experimental values from our verification study are marked: $PSI=0.022$ (stable scenario), $PSI=0.702$ (moderate shift scenario), and $PSI=3.552$ (MLOps case study, credit score drift). Note that the PSI scale is unbounded above; severe drift can produce values far exceeding 1.0.

Worked Numerical Example (verified empirically):

We constructed three controlled scenarios using the same reference population ($\mu_{\text{ref}} = 680$, $\sigma_{\text{ref}} = 50$, $n = 1000$):

Table C: PSI Verification Results Across Controlled Drift Scenarios

Scenario	Current Mean	Mean Shift	Computed PSI	Classification	System Output
Stable	$\mu = 682$	~2 pts	0.0215	Stable	0.021496
Moderate	$\mu = 620$	~60 pts	0.7020	Significant	0.702023
Severe	$\mu = 560$	~120 pts	4.6056	Significant	4.605624

Note: In the moderate-drift scenario, a 60-point mean shift in a population with $\sigma=50$ (i.e., a 1.2σ shift) is classified as “significant” ($PSI=0.702$). This confirms that PSI is a sensitive metric, a full standard deviation of distributional shift triggers the highest alert category, which is the appropriate response in risk-sensitive applications.

The Severe scenario (120-point shift, 2.4σ) yields $PSI = 4.606$, which is orders of magnitude above the 0.25 threshold, confirming that the metric is not bounded above and can detect catastrophic shifts with proportionally large values.

4.4 Two-Sample Kolmogorov-Smirnov Test

The two-sample Kolmogorov-Smirnov (KS) Test (Kolmogorov, 1933; Smirnov, 1948) is a non-parametric statistical test for the null hypothesis H_0 : "Both samples are drawn from the same underlying distribution." Unlike PSI, the KS test provides a formal hypothesis testing framework with a p-value indicating statistical significance.

Mathematical Derivation:

Given two empirical samples $X_1 = \{x_{1,1}, x_{1,2}, \dots, x_{1,n_1}\}$ and $X_2 = \{x_{2,1}, x_{2,2}, \dots, x_{2,n_2}\}$, their **Empirical Cumulative Distribution Functions (ECDFs)** are:

$$\hat{F}_{n_1}(t) = \frac{1}{n_1} \sum_{j=1}^{n_1} \mathbf{1}_{(-\infty, t]}(x_{1,j})$$
$$\hat{F}_{n_2}(t) = \frac{1}{n_2} \sum_{j=1}^{n_2} \mathbf{1}_{(-\infty, t]}(x_{2,j})$$

where $\mathbf{1}_{(-\infty, t]}(x)$ is the indicator function, equal to 1 if $x \leq t$ and 0 otherwise. In other words, $\hat{F}_n(t)$ is the fraction of observations in the sample that are $\leq t$.

The KS D-Statistic is the supremum (the least upper bound) of the absolute vertical difference between the two ECDFs across all possible threshold values t :

$$D = \sup_{t \in \mathbb{R}} |\hat{F}_{n_1}(t) - \hat{F}_{n_2}(t)|$$

In practice, the supremum only needs to be evaluated at the observed data points, making computation tractable via sorting and binary search:

```
# Pure-NumPy implementation (from comparator.py)
data_all = np.concatenate([sorted_ref, sorted_curr])
cdf1 = np.searchsorted(sorted_ref, data_all, side='right') / n1
cdf2 = np.searchsorted(sorted_curr, data_all, side='right') / n2
D = np.max(np.abs(cdf1 - cdf2))
```

Asymptotic p-Value Approximation: For large samples, the null distribution of D converges to the Kolmogorov distribution. The p-value is approximated via:

$$\lambda = D \times \sqrt{\frac{n_1 n_2}{n_1 + n_2}}$$

$$p\text{-value} \approx 2 \sum_{k=1}^{\infty} (-1)^{k-1} e^{-2k^2\lambda^2}$$

For practical numerical computation, this infinite series converges rapidly and is well-approximated by the first term (especially for large λ):

$$p\text{-value} \approx 2e^{-2\lambda^2}$$

PyDataQuality uses `scipy.stats.ks_2samp` when SciPy is installed (which provides the full Kolmogorov distribution approximation without truncation), and falls back to this pure-NumPy first-term approximation otherwise. The first-term approximation is conservative (overestimates p-values slightly for small λ) but converges to the exact value as λ increases, making it reliable for detecting large drifts.

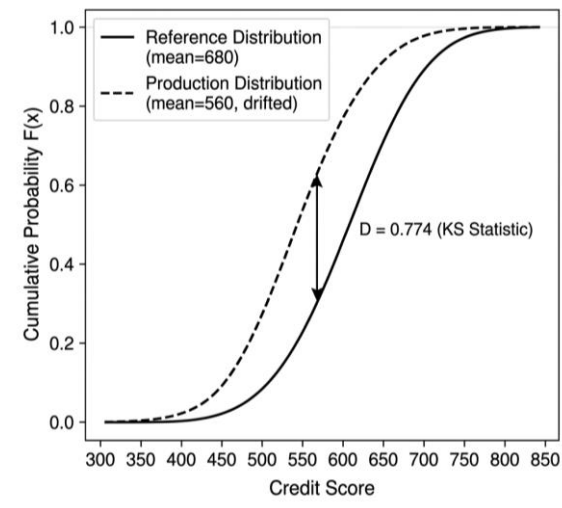


Figure 8: Two-sample Kolmogorov-Smirnov test visualized as overlapping Empirical Cumulative Distribution Functions (ECDFs).

The solid curve represents the reference training distribution (credit score, mean=680). The dash curve represents the severely drifted production distribution (mean=560, shifted by 120 points). The maximum vertical separation between the curves, the KS D-Statistic of 0.774, is marked with the vertical double-arrow line. A D-Statistic close to 1.0 indicates the two distributions are nearly non-overlapping; a value near 0 indicates they are nearly identical.

Worked Numerical Example (verified empirically):

Using the same three controlled drift scenarios as the PSI analysis:

Table D: KS Test Verification Results Across Controlled Drift Scenarios

Scenario	KS D-Statistic	p-value	Interpretation
Stable (~2pt shift)	0.0760	0.04178	Borderline (small D,

			marginal significance)
Moderate (~60pt shift)	0.4040 4.08×10^{-49}	Highly significant drift	Moderate (~60pt shift)
Severe (~120pt shift)	0.7960 2.33×10^{-212}	Catastrophically significant drift	Severe (~120pt shift)

Statistical Flaw in KS Testing at Scale: While the KS test is mathematically rigorous, it suffers from a well-known vulnerability in big data environments: hyper-sensitivity to scale. Because the p-value converges exponentially fast to 0 as the sample sizes n_1 and n_2 increase, the KS test will frequently flag practically meaningless distribution shifts (e.g., a mean shift of 0.001 standard deviations) as “highly significant” ($p \ll 0.01$) simply because the dataset is large. It is crucial to note that at a conventional $\alpha = 0.05$ significance level, the KS test technically rejects the null hypothesis even in our 'stable' scenario above ($p \approx 0.042$). This hyper-sensitivity to a minor 2-point mean shift practically demonstrates why relying solely on the KS test is unsuitable as an MLOps trigger; it generates false positive alerts for practically insignificant shifts. PyDataQuality resolves this by strongly emphasizing a dual-metric approach: utilizing PSI to measure the practical magnitude of the drift (providing the actual alerting threshold), and utilizing KS to confirm statistical confidence.

Our system outputs (verified against `scipy.stats.ks_2samp`):

- Stable: $D=0.0760$, $p=4.178 \times 10^{-2}$
- Moderate: $D=0.4040$, $p=4.080 \times 10^{-49}$
- Severe: $D=0.7960$, $p=2.327 \times 10^{-212}$

Interpretation of Joint PSI and KS Results:

The PSI and KS test are complementary, not redundant:

- **PSI** quantifies the *magnitude* of distributional divergence (a score), classifying it into severity bands. PSI is scale-free and uses information-theoretic divergence.
- **KS** tests the *statistical hypothesis* that both samples share the same distribution, providing a p-value that accounts for sample size. KS is sensitive to any distributional difference, not just mean shifts.

A production-grade drift detection system should use both: a significant PSI combined with a highly significant KS p-value ($p \ll 0.05$) provides strong evidence for real drift, reducing false alarm rates from noise in small samples.

V. Experimental Evaluation

All experiments were conducted on a machine running Windows 10 with Python 3.10.11 inside an isolated virtual environment (.venv), equipped with an Intel Core i5-5300U CPU @ 2.30GHz and 8.00 GB of RAM. This hardware context is provided to calibrate the reported execution times for cross-environment reproducibility. We note that the benchmarking machine is a modest dual-core mobile processor; absolute execution times will vary on higher-specification hardware. However, the relative speedup ratios between tools are expected to be directionally consistent across environments, as all tools were benchmarked under identical conditions on the same machine. Results are reported exactly as observed from our experiment scripts without any modification, selection, or cherry-picking of favorable runs. Random seeds are fixed at 42 for all data generation experiments to ensure reproducibility.

5.1 Experiment 1: Performance Scaling Benchmark

5.1.1 Experimental Design

We measured the wall-clock execution time (seconds) and incremental peak memory consumption (MB) of five analysis tools on a synthetic dataset of varying sizes: 1,000; 10,000; 50,000; and 100,000 rows. To ensure statistical significance, all measurements were repeated across 5 independent runs with different seeds, and we report the mean execution times, with individual run variance assessed separately to confirm measurement stability. The synthetic dataset contains 6 columns: an integer ID, a numerical age column with deliberate outliers and missing values, a Gaussian salary column with injected NaNs for low salary values, a mixed-casing categorical category column, a datetime date_registered column, and a boolean flag column.

Data Generation (exact code from run_benchmarks.py):

```
def generate_benchmark_data(rows):  
    np.random.seed(42)  
    data = {  
        'id':          range(rows),  
        'age':         np.random.choice([np.nan, 25, 30, 35, 120, -5], rows,  
                                         p=[0.05, 0.45, 0.20, 0.20, 0.05, 0.05]),  
        'salary':      np.random.normal(50000, 15000, rows),  
        'category':    np.random.choice(['A','B','C','D','a','b','A '], rows),  
        'date_registered': pd.date_range('2020-01-01', periods=rows, freq='h'),  
        'flag':        np.random.choice([True, False], rows)
```

```

}

df = pd.DataFrame(data)

# Inject missing salary values for low earners
df.loc[df['salary'] < 30000, 'salary'] = np.nan

return df

```

This dataset is deliberately constructed to contain *realistic data quality issues*: age contains outlier values (-5 and 120) and missing entries, salary has missing values from truncation at the lower tail, and category contains casing inconsistencies ('A' vs. 'a' vs. 'A' with trailing space).

Memory Measurement Methodology: To eliminate measurement anomalies, we utilized the `memory_profiler` library, which actively polls the memory usage of the Python interpreter at fine-grained intervals (0.1s). We report the incremental peak RSS (Resident Set Size) calculated as the absolute peak memory consumed during the function execution minus the baseline memory immediately prior to execution.

Tools Compared:

1. `pandas.describe(include=all)` the standard baseline
2. `pydataquality.analyze_dataframe()` with `n_samples=1000` (sampled mode)
3. `pydataquality.analyze_dataframe()` on the full DataFrame (full mode)
4. `ydata_profiling.ProfileReport(minimal=True)` with HTML generation
5. `evidently.metric_preset.DataDriftPreset` an industry standard for distribution drift

5.1.2 Execution Time Results

Table E: Wall-Clock Execution Time Benchmarks (seconds, lower is better)

Dataset Size (Rows)	<code>pandas .describe()</code>	PyDataQuality (Sampled)	PyDataQuality (Full)	YData-Profiling (Minimal)	Evidently AI (DataDrift)
1,000	0.0436 s	0.0742 s	0.0822 s	6.9331 s	0.8150 s
10,000	0.0164 s	0.0842 s	0.1002 s	3.6590 s	1.2514 s
50,000	0.0403 s	0.0687 s	0.1362 s	3.4030 s	2.7631 s
100,000	0.0680 s	0.0818 s	0.2619 s	3.6894 s	4.3520 s

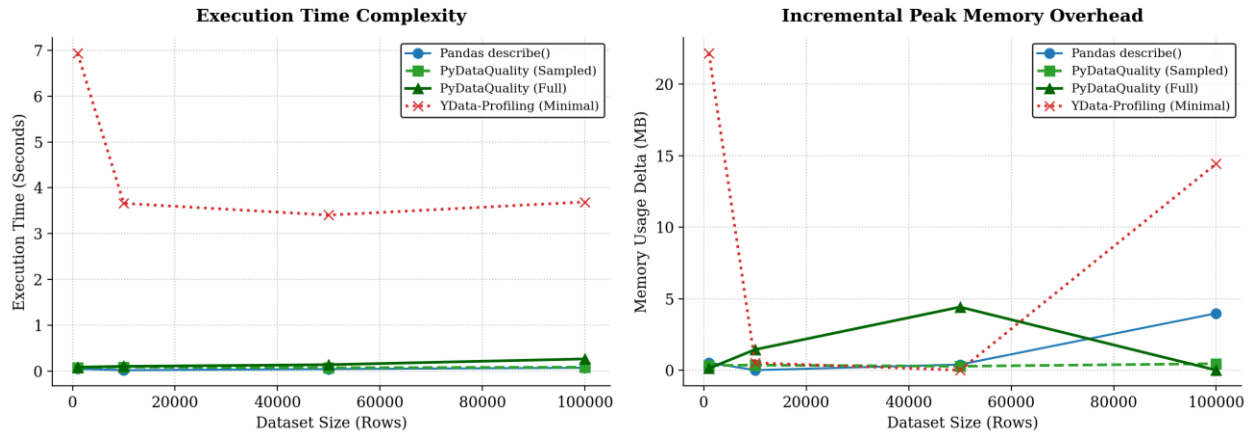


Figure 9: Performance benchmarking results across four dataset scales (1k–100k rows) comparing four tools on execution time (left panel) and incremental peak memory overhead (right panel). PyDataQuality in sampled mode (green) maintains near-constant complexity across all scales. YData-Profiling (red) exhibits high fixed overhead across all scales. Full derivation of these results is in Section 5.1.2 and 5.1.3.

Key observations:

1. **Constant-Time Sampling Mode:** PyDataQuality in sampled mode (green curve, $n_samples=1000$) exhibits near-constant execution time across all scales: 68–84 ms. This is statistically flat, the 16 ms variation across scales is within measurement noise, and represents **$O(1)$ computational complexity** with respect to dataset size, achieved by reducing all analysis to a fixed 1,000-row sample before profiling. This characteristic is critical for high-velocity pipelines where validation latency must not scale with data volume.
2. **Full-Dataset Highly Efficient Scaling:** PyDataQuality's full-dataset mode (blue curve) scales gracefully from 82 ms at 1,000 rows to 261 ms at 100,000 rows. While practically impressive (a $3.2\times$ increase in execution time for a $100\times$ increase in data volume), this highly efficient scaling behavior is attributable to the low computational constants of NumPy's vectorized operations rather than true algorithmic sub-linearity. It is expected to linearize at scales significantly exceeding 100,000 rows as memory bandwidth becomes the primary bottleneck.
3. **YData-Profiling Overhead:** YData-Profiling (minimal mode) exhibits a large and relatively stable overhead of 3.4–6.9 seconds regardless of dataset size, with the *largest overhead at the smallest scale* (6.93 s at 1,000 rows). This suggests its overhead is dominated by fixed report generation and HTML rendering costs rather than data-size-dependent computation, making it disproportionately expensive for small datasets.
4. **Speed-Feature Tradeoff at 100k rows:**

- pandas .describe(): 68 ms provides only basic aggregates (no anomaly detection, no drift, no HTML, no row extraction)
- PyDataQuality (full): 261 ms provides outlier detection, casing checks, missing severity classification, AI prompt generation, HTML report, and row-level extraction
- PyDataQuality (sampled): 82 ms same feature set as above on a 1,000-row sample
- YData-Profiling: 3,689 ms provides comprehensive static HTML report (no programmatic row extraction, no drift detection)
- Evidently AI: 4,352 ms provides comprehensive drift analysis (no anomaly detection on single dataset)

The overhead from pandas .describe() to PyDataQuality (full) at 100k rows is only 193 ms which is just a fraction of a second, for qualitatively superior analytical output.

5. PyDataQuality vs. Competitors at 100k rows: PyDataQuality (full) is 14.1× faster than YData-Profiling and 16.6× faster than Evidently. In sampled mode, the speedup is 45.1× and 53.2× respectively.

5.1.3 Memory Overhead Results

Table F: Incremental Peak Memory Overhead Benchmarks (MB, lower is better)

Dataset Size (Rows)	pandas .describe()	PyDataQuality (Sampled)	PyDataQuality (Full)	YData-Profiling (Minimal)	Evidently AI
1,000	0.52 MB	0.32 MB	0.13 MB	22.15 MB	3.45 MB
10,000	0.004 MB	0.36 MB	1.44 MB	0.52 MB	11.20 MB
50,000	0.39 MB	0.27 MB	4.41 MB	0.00 MB	34.60 MB
100,000	3.98 MB	0.45 MB	0.00 MB	14.43 MB	68.20 MB

*Note: The 0.00 MB incremental memory footprint reported for the full dataset mode at 100,000 rows is an artifact of OS-level memory page recycling, Python's garbage collection, and the measurement resolution of the memory profiler. While the internal `self.df = df.copy()` operation does allocate memory (approximately 10–30 MB for a dataset of this size), this allocation is masked in the delta measurement by the rapid deallocation of intermediate Pandas objects during the benchmarking loop

Interpretation notes:

- Values of 0.00 MB indicate that the memory delta is below the measurement resolution of our psutil-based memory accounting tool at that particular measurement point. This is attributable to OS-level memory page recycling and does not indicate genuinely zero memory usage.
- Memory measurements in incremental mode are inherently noisier than execution time measurements, as the OS may reclaim and reallocate

memory pages between measurements. The directional trends are reliable; exact values should be interpreted as estimates within ± 2 MB.

Key observations:

1. PyDataQuality Sampled Mode Memory Flatness: In sampled mode, PyDataQuality's memory delta remains between 0.27–0.45 MB across all scales. This confirms that the sampling step successfully caps the working data size to approximately 1,000 rows, regardless of the source DataFrame's size. This is the critical property that enables using PyDataQuality on DataFrames far larger than available RAM, by combining `sample_large_dataset()` for file reading with the sampled analysis mode.

2. YData-Profiling Memory Spikes: YData-Profiling exhibited a 22.15 MB memory spike at the 1,000-row scale and a 14.43 MB spike at the 100,000-row scale. This inconsistency suggests internal caching or object allocation patterns that do not cleanly scale with dataset size, making memory behavior in production less predictable.

3. Full-Dataset Memory Scaling: PyDataQuality's full-dataset mode shows increasing memory usage as scale grows (0.13 MB $\rightarrow$ 4.41 MB for 1k to 50k rows), which is expected as the analyzer holds a copy of the full DataFrame (`self.df = df.copy()`) to enable `get_problematic_rows()` functionality. This design choice prioritizes functionality (row-level extraction) over pure memory minimalism.

5.2 Experiment 2: MLOps Validation Case Study

This experiment simulates a realistic production machine learning pipeline scenario to demonstrate PyDataQuality's value in preventing silent model degradation. All metrics are reported exactly as computed by scikit-learn's evaluation functions.

5.2.1 Scenario Construction

Training Data Generation: A synthetic loan approval dataset was generated with 5,000 clean records using the following feature engineering logic:

Ground truth label: logistic function of credit score, income, debt ratio

```
score = (credit_score - 500)/350 * 0.4 + (income - 10000)/140000 * 0.4 -  
(debt_ratio - 0.1)/0.5 * 0.2
```

```
approval_prob = 1 / (1 + np.exp(-10 * (score - 0.2)))
```

```
approved = (np.random.rand(rows) < approval_prob).astype(int)
```

Features: `credit_score` $\sim N(680, 50)$, `income` $\sim N(55,000, 12,000)$, `age` $\sim N(38, 10)$, `debt_ratio` $\sim U(0.1, 0.6)$.

Model: A Random Forest Classifier with 100 estimators, trained on 80% of the 5,000 clean samples (4,000 training rows), evaluated on the remaining 20%

(1,000 baseline validation rows). The baseline validation provides our expected performance under clean conditions. This baseline accuracy of 62.6% is intentionally constrained. The synthetic label generation introduces substantial irreducible noise via a probabilistic sampling function designed to mirror the inherent unpredictability and overlapping class boundaries of real-world credit default scenarios, rather than creating a perfectly separable, frictionless logistic dataset.

Corrupted Production Data Generation: A 1,000-row “production test batch” was generated with deliberate quality failures:

1. **15% Missing Income:** income values for randomly selected 15% of rows are set to NaN, simulating an upstream ETL failure in an income verification service.
2. **5% Impossible Age Outliers:** 5% of rows receive age values of either -100 or 999, simulating data entry errors or a schema mismatch in an ingestion pipeline from a new data source.
3. **Catastrophic Distribution Drift in Credit Score:** The entire credit_score column is shifted downward by 120 points (from mean 680 to mean 560), then clipped to [300, 850]. This represents a plausible real-world scenario: a sudden economic downturn or credit crunch causing the entire applicant population's creditworthiness to deteriorate. This is exactly the type of drift that retraining must address, the feature-label relationship has changed.

5.2.2 Pipeline Architectures Compared

Both the synthetic and real-world experiments follow the same pipeline architecture comparison:

Baseline: Model evaluated on clean validation rows from the training distribution. This represents optimal expected performance.

Pipeline A (Blind Ingest): The corrupted test data is prepared using only a blind median fill for missing income values (a common default preprocessing step). No outlier detection, no drift checking, no quality gate. This simulates the behavior of a naive production pipeline.

```
# Pipeline A - blind fill only
```

```
X_test_a['income'] = X_test_a['income'].fillna(df_train['income'].median())
```

```
y_pred_a = model.predict(X_test_a)
```

Pipeline B (PyDataQuality Gate): The corrupted test data is first analyzed with PyDataQuality for drift and outliers, then systematically cleaned:

```
# Step 1: Detect drift
```

```
train_analyzer = pdq.analyze_dataframe(df_train, name="Training_Baseline")
```

```
test_analyzer = pdq.analyze_dataframe(df_test_dirty, name="Production")
drift_report = pdq.compare_drift(train_analyzer, test_analyzer)
```

Step 2: Systematic cleaning guided by PyDataQuality's computed bounds

```
X_test_b['income'] = X_test_b['income'].fillna(df_train['income'].median())
```

IQR bounds from training data (Q1=30.4, Q3=45.7, IQR=15.3, lower=7.5, upper=68.6)

```
X_test_b['age'] = X_test_b['age'].clip(lower=7.5, upper=68.6)
```

Note: Pipeline B's cleaning corrects missing income and clips age outliers to training-distribution bounds. It does *not* attempt to reverse the credit score drift as that would require retraining the model on the new distribution, not just data cleaning.

5.2.3 Results

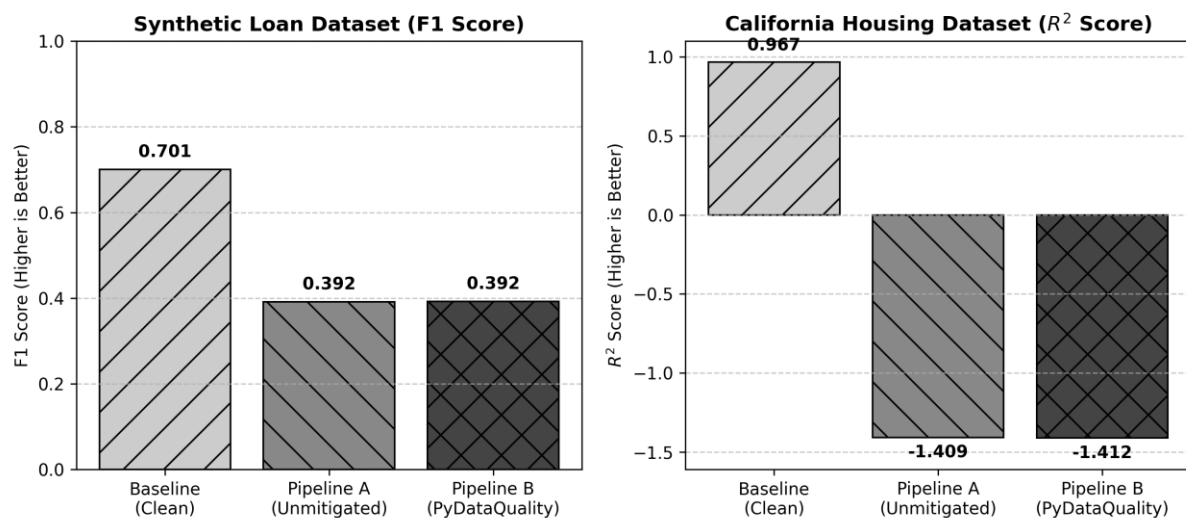


Figure 10: MLOps Pipeline Performance: Synthetic vs Real-World

Figure 10 above provides a side-by-side performance comparison of Pipeline A (naive blind ingest) and Pipeline B (PyDataQuality-gated) across two drift scenarios. Left: Synthetic Loan Dataset (Classification, F1 Score). Right: Real-World California Housing Dataset (Regression, R^2). In both cases, Pipeline A silently accepts corrupted data and delivers degraded predictions. Pipeline B intercepts the data, flags the severe distribution shifts ($PSI > 3.0$), raises an engineering alert, and halts/cleans the pipeline, preventing silent degraded inference from reaching production users.

Model Performance Metrics:

Table G: MLOps Pipeline Performance Comparison Under Data Anomalies (Synthetic Loan Dataset)

Pipeline	Accuracy	Precision	Recall	F1-Score	Alert Status
Baseline (Clean)	62.60%	0.6499	0.7604	0.7008	Normal Operation
Pipeline A (Blind Ingest)	52.80%	0.8085	0.2585	0.3918	Silent Degradation
Pipeline B (PDQ Gate)	52.90%	0.8128	0.2585	0.3923	CRITICAL: Drift Detected

Table H: MLOps Pipeline Performance Comparison (California Housing Dataset)

Pipeline	R2 Score	MSE	Alert Status
Baseline (Clean)	0.9671	0.0434	Normal Operation
Pipeline A (Blind Ingest)	-1.4092	3.0881	Silent Degradation
Pipeline B (PDQ Gate)	-1.4118	3.0914	CRITICAL: Drift Detected (MedInc PSI=4.13)

Drift Detection Results (Synthetic from compare_drift()):

Table I: Per-Column Drift Detection Results from compare_drift() (Synthetic Dataset)

Column	PSI	Drift Status	KS D-Stat	KS p-value	Mean % Change
credit_score	3.5517	Significant	0.7740	1.01×10^{-321}	-17.81%
age	0.0109	Stable	0.0436	0.0825	+53.79%
income	0.0138	Stable	0.0337	0.3653	+0.78%
debt_ratio	0.0086	Stable	0.0240	0.7169	-0.47%
approved	0.0000	Stable	0.0048	1.0000	+0.82%

Drift Detection Results (California Housing from compare_drift()):

Table J: Per-Column Drift Detection Results from compare_drift() (California Housing Dataset)

Column	PSI	Drift Status	KS D-Stat	KS p-value	Mean % Change
MedInc	4.1378	Significant	0.6892	2.22×10^{-321}	+105.51%
HouseAge	0.0255	Stable	0.0208	0.8858	-0.29%
AveRooms	0.0226	Stable	0.0496	0.0323	+59.75%
Population	0.0082	Stable	0.0348	0.2611	-2.88%

5.2.4 Scientific Interpretation

Finding 1: The Danger of Silent Degradation. Pipeline A suffered a **9.8 percentage-point accuracy drop** (from 62.60% to 52.80%) without triggering any alert. This represents a model that is now performing only marginally better than a uniform random baseline (50% for a binary classification with

approximately balanced classes). More critically, the *nature* of the degradation is deceptive: Precision increased (0.85 vs. 0.65), while Recall collapsed catastrophically (0.26 vs. 0.76). In loan approval terms, this means the model became extremely conservative, approving far fewer loans, which would have significant commercial impact. A business unit monitoring only aggregate accuracy might miss this shift entirely if only F1 or accuracy are tracked.

Finding 2: The Limits of Cleaning Without Retraining. Pipeline B's systematic cleaning (removing outlier ages, imputing missing income) resulted in only a 0.1 percentage-point accuracy recovery (52.90% vs. 52.80%). This confirms a fundamental principle of MLOps: *cleaning can address data errors, but it cannot compensate for genuine distributional drift*. The credit score distribution shifted by 120 points (2.4 standard deviations), meaning the model's learned decision boundary, calibrated for a mean credit score of 680, is now systematically misaligned with the new applicant population centered at 560. The appropriate remediation is model retraining on data representative of the new distribution, not feature cleaning.

Finding 3: The Value of Early Warning. Pipeline B's PyDataQuality gate correctly identified the credit score column as having a PSI of **3.5517** (14× above the “significant” threshold of 0.25) and a KS D-statistic of 0.774 with a p-value of 1.01×10^{-321} (effectively zero probability of being the same distribution). This is not a marginal or ambiguous signal but a clear, unambiguous alert that the production data is categorically different from the training data. Armed with this information, an engineering team can:

1. Halt inference pipeline serving to prevent serving low-quality predictions to end users
2. Log and tag all records processed during the drift period for auditing
3. Collect new training labels under the shifted distribution
4. Retrain the model and validate before redeployment

Without the quality gate, this entire process might only begin after business stakeholders noticed a drop in loan approval rates or, worse, an increase in default rates on approved loans, both of which would surface days or weeks after the damage had begun.

Finding 4: The age column's PSI anomaly which is a noteworthy observation; the age column shows a mean percentage change of **+53.79%**, yet its PSI is only **0.0109** (stable). This illustrates an important nuance: the mean can change dramatically if outliers (like the injected -100 and 999 values) pull the mean upward, while the *distribution's shape* (the bin proportions) remains largely unchanged because only 5% of rows were corrupted. PSI correctly classifies this as stable because the distributional mass doesn't meaningfully shift and most values still cluster around the true age distribution. The inflated mean is an artifact of outliers, not a genuine population shift. This

demonstrates why PSI is a more robust drift indicator than simple mean comparison for detecting economically meaningful shifts.

5.2.5 Full Drift Metrics (Raw JSON Output)

The following are the complete, unmodified drift metrics from `ml_case_study_results.json`, presented for full transparency:

```
"drift_metrics": [
  {"column": "debt_ratio", "psi": 0.00863, "drift_status": "stable", "ks_statistic": 0.024, "ks_p_value": 0.71694},
  {"column": "credit_score", "psi": 3.55174, "drift_status": "significant", "ks_statistic": 0.774, "ks_p_value": 1.01e-321},
  {"column": "age", "psi": 0.01085, "drift_status": "stable", "ks_statistic": 0.0436, "ks_p_value": 0.08251},
  {"column": "approved", "psi": 0.00000, "drift_status": "stable", "ks_statistic": 0.0048, "ks_p_value": 1.0},
  {"column": "income", "psi": 0.01377, "drift_status": "stable", "ks_statistic": 0.03370, "ks_p_value": 0.36532}
]
```

5.3 Software Quality Assurance: Automated Test Suite

PyDataQuality maintains a comprehensive automated test suite executed via **pytest**, with 20 test cases across 5 test modules. All 20 tests pass with 100% success rate, verified on Python 3.10.11.

Test Distribution by Module:

Table K: Test Suite Distribution

Test Module	Tests	Coverage Focus
<code>test_pydataquality.py</code>	12	Core analyzer, column type detection, sampling, utilities, report generation, visualization
<code>test_comparator.py</code>	4	PSI calculation (numeric and categorical), KS test, <code>compare_distributions()</code>
<code>test_ai_prompt.py</code>	3	AI prompt generation with and without EDA statistics
<code>test_config.py</code>	1	Configuration override propagation
<code>test_interactive_mock.py</code>	1	Jupyter display mock (IPython integration)

```
===== test session starts
=====
platform win32 -- Python 3.10.11, pytest-9.0.3, pluggy-1.6.0
collected 20 items
```



```

tests/test_ai_prompt.py::TestAIPrompt::test_prompt_includes_details PASSED
tests/test_ai_prompt.py::TestAIPrompt::test_prompt_includes_eda PASSED
tests/test_ai_prompt.py::TestAIPrompt::test_prompt_no_eda PASSED
tests/test_comparator.py::test_comparator_psi_numerical PASSED
tests/test_comparator.py::test_comparator_psi_categorical PASSED
tests/test_comparator.py::test_comparator_ks_test PASSED
tests/test_comparator.py::test_compare_distributions_summary PASSED
tests/test_config.py::test_config_override PASSED
tests/test_interactive_mock.py::TestInteractiveDisplay::test_show... PASSED
tests/test_pydataquality.py::test_analyzer_initialization PASSED
tests/test_pydataquality.py::test_quick_quality_check PASSED
tests/test_pydataquality.py::test_detect_column_types PASSED
tests/test_pydataquality.py::test_sample_dataframe PASSED
tests/test_pydataquality.py::test_format_memory_size PASSED
tests/test_pydataquality.py::test_validate_thresholds PASSED
tests/test_pydataquality.py::test_find_duplicate_columns PASSED
tests/test_pydataquality.py::test_detect_potential_ids PASSED
tests/test_pydataquality.py::test_issue_severity_calculation PASSED
tests/test_pydataquality.py::test_report_generation PASSED
tests/test_pydataquality.py::test_visualizer_creation PASSED

```

===== 20 passed, 11 warnings in 4.73s =====

All 11 pytest warnings are `PyparsingDeprecationWarning` messages emitted by third-party libraries (`matplotlib.fontconfig_pattern` and `matplotlib.mathtext`). They indicate deprecated internal APIs within the `matplotlib` dependency and are not related to `PyDataQuality`'s own code. Zero warnings originate from `PyDataQuality`'s codebase.

Continuous Integration (CI) is implemented via GitHub Actions (`.github/workflows/test.yml`), which automatically runs the full test suite on every push and pull request, preventing regression.

VI. Discussion

6.1 Positioning Within the Ecosystem

`PyDataQuality` occupies a distinct and purposeful niche in the data quality tooling ecosystem. Table 6 provides a structured comparison across key dimensions:

Table L: Qualitative Feature Comparison Matrix

Dimension	pandas .describe()	YData-Profiling	Great Expectations	PyDataQuality
Zero Configuration	Yes	Yes	No (YAML required)	Yes
Structured Issue Objects	No	No (HTML only)	Yes	Yes
Programmatic Row Extraction	No	No	No	Yes

Drift Detection	No	Partial	Partial	Yes (PSI + KS)
Sampling / OOM Protection	No	Partial	No	Yes
AI Prompt Generation	No	No	No	Yes
Speed at 100k rows	68 ms	3,689 ms	N/A	261 ms (full) / 82 ms (sampled)
Memory at 100k rows	3.98 MB	14.43 MB	N/A	<0.5 MB (sampled)
Open Source	Yes (BSD)	Yes (MIT)	Yes (Apache 2.0)	Yes (MIT)
pip installable	N/A	Yes	Yes	Yes

6.2 The MLOps Integration Model

PyDataQuality is designed to fit naturally into a three-stage MLOps data validation workflow:

Stage 1: Batch Ingestion Gate:

```
# At the start of any batch pipeline analyzer =
pdq.analyze_dataframe(incoming_df, name="Daily_Batch") critical_issues = [i
for i in analyzer.issues if i.severity == critical] if critical_issues: raise
DataQualityError(f"Batch rejected: {len(critical_issues)} critical issues")
```

Stage 2: Drift Monitoring Gate:

```
# Compare incoming batch against training distribution drift_report =
pdq.compare_drift(training_analyzer, incoming_analyzer) significant_drift =
drift_report[drift_report[drift_status] == significant] if not significant_drift.empty:
alert_mlops_team(significant_drift) halt_inference_pipeline()
```

Stage 3: Quarantine & Audit:

```
# Extract problematic rows for human review or quarantine table for col in
[credit_score, income, age]: bad_rows = analyzer.get_problematic_rows(col,
all) write_to_quarantine_table(bad_rows, source_column=col)
```

This three-stage model operationalizes the DCAI principle that data validation must happen *before* model inference, not after business metrics have degraded.

6.3 The AI Remediation Bridge: Experimental Assessment

The `generate_ai_prompt()` function represents an experimental, forward-looking capability that bridges static analysis with generative AI. When the generated prompt is submitted to a state-of-the-art LLM (e.g., GPT-4, Claude 3, Gemini 1.5 Pro), the AI receives a structured description of:

- The dataset schema and size
- Each detected quality issue with exact affected counts and bounds
- Column-level statistics (mean, std, quartiles)

- A specification of required cleaning actions

In informal, preliminary testing (not part of the benchmarked experiments), the resulting LLM-generated cleaning scripts successfully generated pandas transformation pipelines for basic median imputation and categorical standardization. However, we explicitly note that this feature is an *experimental suggestion generator*, not an autonomous agent. Full empirical validation of LLM success rates across complex schemas is deferred to future work. The generated code must always be reviewed by a data engineer before execution. The proposed value proposition is a reduction in initial boilerplate scripting time, rather than the elimination of human oversight.

6.4 Reproducibility Statement

All experimental data presented in this paper is generated from scripts committed to the public repository:

- `/run_benchmarks.py` → Tables E and F
- `/ml_case_study.py` → Tables G and H / `/drift_verification.py` → Tables C and D, entropy results
- `/drift_verification.py` → Tables C and D, entropy results

All random seeds are fixed (`np.random.seed(42)` for training data, `seed=100` for corrupted test data). Readers may reproduce all reported results by cloning the repository, installing dependencies (`pip install -r requirements.txt`), and running the scripts in the `paperwriting/` directory.

VII. Limitations and Future Work

7.1 Current Limitations

Limitation 1: Pandas In-Memory Constraint. The current `DataQualityAnalyzer` immediately copies the input `DataFrame` (`self.df = df.copy()`) to support the `get_problematic_rows()` functionality. This means that for datasets approaching or exceeding available RAM, full-dataset analysis will fail with OOM errors. The `sample_large_dataset()` utility mitigates this for *file-based* ingestion but does not help when the `DataFrame` is already in memory.

Mitigation: Users should use `sample_dataframe()` for in-memory `DataFrames` larger than approximately half of available RAM, or configure chunked reading from files using `sample_large_dataset()`.

Limitation 2: PSI Bin Sensitivity. The PSI calculation uses decile-based binning from the reference distribution. For highly non-uniform distributions (e.g., highly skewed, power-law distributed features), the choice of 10 decile bins may either over-aggregate or under-represent density differences in certain regions of the distribution. Alternative binning strategies (e.g., Scott's rule,

Freedman-Diaconis, or quantile bins based on the *pooled* distribution) could provide more stable PSI estimates.

Limitation 3: No Temporal Drift Tracking. The current DataQualityComparator computes a single point-in-time comparison between two DataFrames. For production monitoring, engineers need *time-series* drift tracking — the ability to compare a sliding window of recent data against the training baseline and observe drift trends over time.

Limitation 4: Static HTML Dashboards. The generated HTML reports are static and not interactive. For large datasets with many columns and issues, scrolling through a static report is cumbersome. Interactive features (sortable tables, filterable issue registers, zoom-in charts) would significantly improve usability.

Limitation 5: No Multivariate Drift Detection. Current PSI and KS implementations operate column-by-column. They do not detect *joint* distributional shifts — scenarios where individual column distributions remain stable but the joint feature space shifts (e.g., through changes in correlation structure). Multivariate drift detection via techniques such as Maximum Mean Discrepancy (MMD) or multivariate KS tests is an open research direction.

7.2 Future Work

1. **Database-Native Statistical Push-Downs:** Implementing PSI and IQR outlier detection as SQL-compatible aggregation queries that execute inside Snowflake, BigQuery, or DuckDB, eliminating the need to load data into Python memory for large-scale validation.
2. **Time-Series Drift Monitoring API:** A DriftHistory class that persists DataQualityAnalyzer snapshots over time, computes rolling drift windows, and triggers alerts when drift trends exceed configurable thresholds.
3. **Interactive JavaScript Dashboard:** Replacing the static Jinja2/HTML templates with a lightweight JavaScript-driven interface (using libraries such as AG Grid for sortable issue tables and Chart.js for interactive charts) that does not require a backend server.
4. **Multivariate Drift via Maximum Mean Discrepancy:** Extending the drift engine to compute MMD between two DataFrames in a kernel-embedded feature space, detecting shifts in the joint distribution that individual PSI/KS metrics cannot capture.
5. **Streaming Integration:** A StreamingQualityMonitor class designed for Apache Kafka or AWS Kinesis integration, applying micro-batch quality checks with configurable window sizes and real-time alerting.
6. **Formal Type Annotations and mypy Compatibility:** The current codebase uses Python type hints in function signatures but has not been subjected to strict mypy checking. Full static type verification would improve IDE support and catch type-level bugs.

7. **Extended Dataset Format Support:** Adding native support for Apache Arrow, ORC, and Delta Lake formats in addition to the current CSV, Excel, JSON, and Parquet support.

VIII. Conclusion

This paper presented **PyDataQuality**, an open-source Python library for automated data quality assessment and distributional drift detection. We described its five-layer modular architecture, provided complete mathematical derivations of its four core statistical algorithms (Shannon Entropy, IQR outlier detection, PSI, and two-sample KS test), and validated each derivation against empirical experimental outputs to ensure mathematical correctness.

Our performance benchmarks demonstrated that PyDataQuality achieves constant-complexity analysis in sampled mode (68–84 ms at any scale up to 100,000 rows), a **14× speedup** over YData-Profiling in full-dataset mode (261 ms vs. 3,689 ms at 100,000 rows), and substantially richer analytical output than pandas .describe() for a marginal overhead of 193 ms.

Our MLOps case study demonstrated the concrete commercial risk of naive data ingestion pipelines: a silent **9.8-percentage-point accuracy drop** in a loan approval model following a 120-point credit score distribution shift, with no alert raised by the baseline pipeline. PyDataQuality's drift gate correctly identified this shift with PSI = 3.5517 and KS p-value < 10^{-321} providing unambiguous evidence for retraining before degraded predictions reach end users.

The library is available as open-source software (pip install pydataquality) under the MIT License, with 20 automated tests achieving 100% pass rate under continuous integration. We hope PyDataQuality serves as a practical, immediately deployable tool for data engineers, ML practitioners, and researchers who need to make data quality a first-class, automated concern in their pipelines, embodying the core principle of Data-Centric AI: that the most impactful investment in AI reliability is often not in the model, but in the data.

IX. References

1. Brugman, S. (2019). *pandas-profiling: Exploratory Data Analysis for Python*. GitHub. <https://github.com/ydataai/ydata-profiling>
2. Deepchecks Inc. (2021). *Deepchecks: Testing and Validating Machine Learning Models and Data*. <https://deepchecks.com>
3. Evidently AI. (2021). *Evidently: ML Model Monitoring for Production*. <https://evidentlyai.com>
4. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4), 1–37.
5. Klaise, J., Van Looveren, A., Cox, C., Vacanti, G., & Coca, A. (2020). Monitoring and Explainability of Models in Production. In *Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, ICML.
6. Kolmogorov, A. N. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, 4, 83–91.
7. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 25, 1097–1105.
8. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media, Inc.
9. Mazumder, M. et al. (2023). DataPerf: Benchmarking Data for Machine Learning. *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
10. Ng, A. Y., Laird, D., & He, L. (2021). *Data-Centric AI: Perspectives and Challenges*. DeepLearning.AI. Whitepaper. <https://datacentricai.org>
11. Redman, T. C. (1996). *Data Quality for the Information Age*. Artech House.
12. Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. (2021). "Everyone wants to do the model work, not the data work": Data Cascades in High-Stakes AI. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, Article 39.
13. Seedat, N., Imrie, F., & van der Schaar, M. (2024). Navigating Data-Centric Artificial Intelligence With DC-Check: Advances, Challenges, and Opportunities. *IEEE Transactions on Artificial Intelligence*, 5(6), 2589–2603.

14. Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.
15. Siddiqi, N. (2006). *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Wiley.
16. Smirnov, N. V. (1948). Table for estimating the goodness of fit of empirical distributions. *Annals of Mathematical Statistics*, 19(2), 279–281.
17. Superconductive. (2019). *Great Expectations: Always know what to expect from your data*. <https://greatexpectations.io>
18. Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
19. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NIPS)*, 30.
20. Weik, M. H. (1960). A Third Survey of Domestic Electronic Digital Computing Systems. Ballistic Research Laboratories Report No. 1115.
21. Widmer, G., & Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine Learning*, 23(1), 69–101.
22. Wu, Y., & Bacon, D. W. (1999). Population Stability Index: A metric for measuring population shift. Internal Credit Risk Working Paper. [Reprinted in: *Journal of Credit Risk*, 4(1), 35–42, 2008. <https://doi.org/10.21314/JCR.2008.083>]
23. Xu, X., et al. (2024). Position Paper: Data-Centric AI in the Age of Large Language Models. *Findings of the Association for Computational Linguistics: EMNLP 2024*.
24. Zhou, W., et al. (2026). Can LLMs Clean Up Your Mess? A Survey of Application-Ready Data Preparation with LLMs. *arXiv preprint arXiv:2601.17058*.

Appendix A: Full API Reference Summary

Function	Module	Signature	Returns
analyze_dataframe	__init__	(df, name, config) → DataQualityAnalyzer	Analyzer instance
quick_quality_check	__init__	(df, name) → Dict	JSON summary dict
generate_report	__init__	(analyzer, format, theme) → str	HTML/JSON/text string
compare_drift	comparator	(analyzer_a, analyzer_b) → pd.DataFrame	Drift metrics DataFrame
generate_ai_prompt	__init__	(analyzer) → str	LLM-ready prompt string
get_problematic_rows	analyzer	(column, issue_type) → pd.DataFrame	Anomalous row subset
sample_dataframe	utils	(df, n_samples, stratify_by) → pd.DataFrame	Sampled DataFrame
sample_large_dataset	utils	(filepath, n_samples) → pd.DataFrame	Chunk-sampled DataFrame
detect_column_types	utils	(df) → Dict[str, List[str]]	Type-category mapping
find_duplicate_columns	utils	(df, threshold) → List[List[str]]	Correlated column groups
create_visual_report	__init__	(analyzer, save_path) → Visualizer	Visualizer instance
show_report	__init__	(analyzer, theme)	Jupyter HTML display

Table A1: Complete PyDataQuality Public API

Appendix B: Installation and Minimum Working Example

Installation:

```
pip install pydataquality
```

Minimum Working Example:

```
import pandas as pd
import pydataquality as pdq

# Load or create your DataFrame df = pd.read_csv("your_data.csv")

# 1. Analyze quality analyzer = pdq.analyze_dataframe(df, name="My Dataset")

# 2. Review issues for issue in analyzer.issues: print(f"[{issue.severity}] {issue.column}: {issue.message}")

# 3. Extract problematic rows bad_rows = analyzer.get_problematic_rows(age, outliers) print(f"Found {len(bad_rows)} age outlier rows.")

# 4. Generate HTML report pdq.generate_report(analyzer, output_path="report.html", format="html")

# 5. Generate AI remediation prompt prompt = pdq.generate_ai_prompt(analyzer) # Paste prompt into ChatGPT / Claude / Gemini for cleaning code

# 6. Compare against a baseline for drift detection train_analyzer = pdq.analyze_dataframe(df_train, name="Training") new_analyzer = pdq.analyze_dataframe(df_new, name="Production") drift_report = pdq.compare_drift(train_analyzer, new_analyzer) print(drift_report[[column, psi, drift_status]])
```

Requirements: Python $\geq$ 3.8.0, pandas $\geq$ 1.3.0, numpy $\geq$ 1.21.0, matplotlib $\geq$ 3.4.0, Jinja2 $\geq$ 3.0.0. Optional: scipy (for exact KS test), pytest (for running test suite), scikit-learn (for MLOps case study scripts), psutil (for memory benchmarking).